



NATIONAL UNIVERSITY
OF COMPUTER AND EMERGING SCIENCES
CHINIOT - FAISALABAD CAMPUS



Programming Fundamentals Lab #14

Lab Instructor(s)	Mr. Ali Raza
Semester	Fall 2023

FAST School of Computing

Instructions:

1. Make a word document with the convention "SECTION_ ROLLNO_L2" and put all your source code and snapshots of its output in it.
2. Plagiarism is strictly prohibited, if you take a code snippet off the internet, mention its reference.
3. Do not discuss solutions with one another.
4. Make a folder (named: 22FXXXX_Section) with all your CPP files and Word file.
5. Submit word file and source file's folder on GOOLE CLASSROOM.

EXAMPLE

C++ program to print a 2D array using a for loop.

```
#include <iostream>
using namespace std;

int main()
{
    int arr[3][3] = {{1, 2, 3}, {4, 5, 6}, {7, 8, 9}};

    for (int i = 0; i < 3; i++)
    {
        for (int j = 0; j < 3; j++)
        {
            cout << arr[i][j] << " ";
        }
        cout << endl;
    }

    return 0;
}
```

Task 01:

Given a 2-Dimensional array, find the largest element in it.

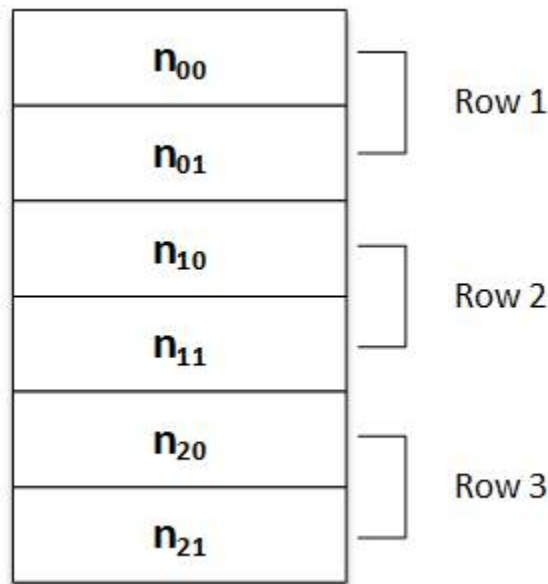
Restriction: You can change the column index while accessing the array but row index can't be changed. If you array's name is "arr[][]" then arr[0][col] can be used to access the array, row must always remain zero, the column numbers can be changed.

I have heard that 2-Dimensional arrays don't exist, 2D arrays are actually implemented as 1D arrays.

Row Major Ordering

C++ stores 2D arrays in row major ordering. In row major order the elements are stored row by row. First row, second row and so on.

For `int number[3][2]`; Row major order will look like:



Like linear array, system keeps track of the address of first element only *i.e. the base address all of the array*.

Using this base address, the computer computes the address of the element in the i^{th} row and j^{th} column, i.e. `loc a[i][j]`, using the formula:

$$\text{loc } a[i][j] = \text{base}(a) + w(n \times i + j)$$

where $\text{base}(a)$ is base address of array a , m is number of rows in array a , n is number of column in array, w is the number of bytes per storage location for one element of the array.

Task 02:

Write a C++ Program to Find the Transpose of a Matrix. Decide dimensions of a rectangular 2D array and take input from the user. Now store its transpose in another array and print its output on screen.

Task 03:

Write a program that declares a 2D character array with dimensions fixed by you. Then, prompt the user to enter strings to fill the array. Finally, print out the contents of the array by converting the uppercase letters into lowercase (If any).

J	a	v	a	\0					
P	y	t	h	o	n	\0			
C	+	+	\0						
H	T	M	L	\0					
S	Q	L	\0						

Task 04:

Write a C++ program to change the numbers in the diagonals of a square matrix to their squares. The program should accept an input square matrix of size $N \times N$, where N is the number of rows and columns, and output the modified matrix with the diagonal elements squared.

The program should meet the following requirements:

The program should only use static arrays, without using any dynamic memory allocation.

The program should not use any built-in or library functions to perform the diagonal modification.

The output should match exactly like the one given below:

```
Microsoft Visual Studio Debug Console

1  2  3  4
5  6  7  8
9 10 11 12
13 14 15 16

After processing:
1  2  3  4
5 36  7  8
9 10 121 12
13 14 15 256

C:\Users\Mughees\source\repos\Project8\x64\Debug\Project8.exe (process 16916) exited with code 0.
To automatically close the console when debugging stops, enable Tools->Options->Debugging->Automatically close the console when debugging stops.
Press any key to close this window . . .
```

Restriction: You are only allowed to use ONE for loop to change the elements in the diagonal.

Task 05:

Now you may have noticed that in question 4, using set width enhanced the readability of the output. Your task is to take input of a 4*4 2D int matrix from the user and then print its values on screen BUT output should automatically adjust according to the values of the array.

Without set width

```
1 2 3 4
5 36 7 8
9 10 121 12
13 14 15 256
```

With set width

```
1  2  3  4
5  36  7  8
9  10 121 12
13 14 15 256
```

Task 06:

Given a 2D-array of integers numbers and an integer *target*, return *indices* of the two elements of the array such when added, the sum of the elements equals to the value of the *target*. You may assume that each input would have exactly one solution, and you may not use the same element twice.

Input: nums = { [0,7,11], [1,5,2], [4,6,10] } target = 9

Output: [0,1] & [1,2]

Explanation: Because nums[0][1] + nums[1][2] == 9, we return [0,1] & [1,2]

If a solution does not exist, print not possible with 2 numbers.

“A language that doesn't affect the way you think about programming is not worth knowing.” — Alan J. Perlis